

```

print len(dict1)
pickle_file = open("mohit.dup1", "w")
cPickle.dump(dict1, pickle_file)
pickle_file.close()
t2= datetime.now()
total =t2-t1
print "Time taken to create ", total

```

The following function opens the pickle file and loads the dictionary into the memory (RAM):

```

def file_open():
    pickle_file = open("mohit.dup1", "r")
    file_dict = cPickle.load(pickle_file)
    pickle_file.close()
    return file_dict

```

The `file_search()` function in the following code is used to match the hash of the file provided by the user. It first opens the pickle file and loads the Python default dictionary. When you provide the file with the pathname, it calculates the MD5 hash of the file and then matches it with the dictionary's keys.

```

def file_search(file_name):
    t1= datetime.now()
    try:
        file_dict = file_open()
    except IOError:
        create()
        file_dict = file_open()
    except Exception as e :
        print e
        sys.exit()
    file_name1 = file_name.rsplit("\\",1)
    os.chdir(file_name1[0])

    file_to_be_searched = file_name1[1]
    if os.access(file_name, os.F_OK):
        if os.access(file_name, os.R_OK):
            sign = md5(file_to_be_searched)
            files = file_dict.get(sign, None)
            if files:
                print "File(s) are "
                files.sort()
                for index, item in enumerate(files):
                    print index+1, " ", item

            print "-----"
            else :
                print "File is not present or accessible"
            t2= datetime.now()
            total =t2-t1
            print "Time taken to search ", total

```

Shown below is the `main()` function responsible for all actions. We will discuss all its options later with diagrams.

```

def main():
    parser.add_argument("file_name", nargs='?',
        help="Give file with path in double quotes")

    parser.add_argument('-c', nargs='?', help="For
        creating MD5 hash of all files", const=4096, type=int)
    parser.add_argument('-a', help="To get all
        duplicate files in duplicate.txt in running current folder",
        action='store_true')
    parser.add_argument('-f', help="To find the MD5
        hash, provide file with path in double quotes ", nargs=1,)

    args = parser.parse_args()
    try:
        if args.c:
            print args.c
            create(args.c)

        elif args.a :
            file_dict = file_open()
            all_duplicate(file_dict)

        elif args.f :
            if os.access(args.f[0], os.R_OK):
                print "Md5 Signature are : ", md5(args.f[0], 4096)
                print "\n"
            else :
                print "Check the file path and file name\n"
            else:
                file_search(args.file_name)
                print "Thanks for using l4wisdom.com"
                print "Email id mohitraj.cs@gmail.com"
                print "URL: http://l4wisdom.com/finder_go.php"

    except Exception as e:
        print e
        print "Please use proper format to
        search a file use following instructions"
        print "dupl file-name"
        print "Use <dupl -h > For help"

```

Let us save the complete code as `dupl.py` and make it a Windows executable (exe) file using the Pyinstaller module. You can also download a readymade exe file from http://l4wisdom.com/dupl_go.php. Run the command as shown in Figure 2. After running it successfully, you can find the `dupl.exe` in folder `C:\Pyinstaller-2.1\dupl\dist`, as shown in Figure 3. You can put the `dupl.exe` file in the Windows folder, but if you place this in a different folder, you will have to set the path to that folder.